

Introduction

- **Définition** : Raisonnement basé sur la forme des énoncés, indépendant du contenu.
- **Intérêt** : Vérifie qu'un programme respecte sa spécification, évite les bugs, plus fiable que tests ou model-checking.
- **Méthode** : Utilisation de systèmes formels (λ -calcul, combinateurs, λ -calcul typé, OCaml) + correspondance avec logique (Curry-Howard, méthode B).
- **Grands noms** : Peano, Frege, Russell, Schönfinkel, Curry, Church.

Chapitre 1 : Le λ -calcul

1. **Introduction** :
 - Système formel de Church (années 1930) pour exprimer tous les calculs via des fonctions.
 - Modèle universel de calcul, base des langages fonctionnels (OCaml, Haskell, Lisp, Python).
 - Tout est fonction (nombres, listes, opérations).
 - Fonctions vues de manière intensionnelle (définition) vs extensionnelle (résultat).
2. **Définition formelle** :
 - Variables : $x, y, z \in \mathcal{V}$
 - λ -termes : variable, application ($M N$), abstraction ($\lambda x . M$)
 - Exemples : $\lambda x.x$ (identité), $\lambda x.(\lambda y. x y)$, composition $\lambda g.\lambda f.\lambda x.g(f(x))$
3. **Conventions pratiques** :
 - Parenthèses externes omises, applications associées à gauche, abstractions regroupées : $\lambda x y z. M$
 - Notation : variables minuscules, λ -termes majuscules
4. **Longueur d'un λ -terme** :
 - Variable : 1, application : somme des longueurs, abstraction : $\text{longueur}(M)+1$
5. **Variables libres et liées** :
 - $FV(M)$: variables libres, $BV(M)$: variables liées
 - Terme fermé = combinateur
6. **α -équivalence** :
 - Deux termes équivalents si seuls les noms de variables changent
 - Substitution $M[x:=N]$: remplace occurrences libres de x par N , attention aux captures
7. **β -réduction** :
 - β -redex : $(\lambda x . M) N \rightarrow M[x:=N]$
 - Forme β -normale = aucun redex
 - Réduction complète : plusieurs étapes possibles
8. **β -abstraction** :
 - Inverse de β -réduction, crée des β -redex à partir d'un λ -terme
9. **β -équivalence** :
 - $M =_{\beta} N$ si passage via β -réduction et/ou β -abstraction
 - Propriétés : réflexivité, symétrie, transitivité, congruence
10. **Stratégies de réduction** :
 - Normale : redex externes/gauche en premier
 - Applicative : redex internes/gauche en premier
 - Redex externe : pas dans un autre redex, interne : contenu dans M ou N
11. **η -réduction** :

- $\lambda x . M \rightarrow_{\eta} M$ si $x \notin FV(M)$
- Forme $\beta\eta$ -normale = forme β -normale + forme η -normale

12. Théorème de Church-Rosser :

- $M \rightarrow N$ et $M \rightarrow P \Rightarrow \exists Z$ tel que $N \rightarrow Z$ et $P \rightarrow Z$
- Forme normale unique (à α près) si elle existe

13. Forme normale :

- Unicité et existence non garanties
- Fortement normalisable : toutes réductions finies
- Faiblement normalisable : au moins une réduction finie
- Non normalisable : réduction infinie

14. Programmation en λ -calcul :

- **Booléens** : $T \equiv \lambda x y.x$, $F \equiv \lambda x y.y$, NOT, AND, IF
- **Entiers de Church** : $N0 \equiv \lambda f x.x$, $N1 \equiv \lambda f x.f x \dots$
- **Opérations** : SUCC, ADD, MULT
- **Paires** : PAIR $\equiv \lambda z.z M N$, FIRST, SCND

Chapitre 2 : Théorie des combinateurs

1. Introduction :

- Système formel sans variables (Schönfinkel, Curry) $\rightarrow$ pas de substitution complexe.
- Combinateur : constante atomique ou application de combinateurs.
- Application associative à gauche : $(A B C) = ((A B) C)$.

2. Correspondance λ -calcul :

- Combinateur = λ -terme sans variables libres.
- Représentation de λ -termes avec un nombre fini de constantes atomiques (ex : S, K, I).

3. Éléments de traduction :

- Remplacer λ -abstractions par combinateurs S, K, I.
- Objectif : $\lambda x . M, \lambda x . M x, \lambda x . \lambda y . N M \rightarrow$ combinateurs.

4. Combinateurs SKI :

- $S \equiv \lambda f g x . f x (g x)$, $K \equiv \lambda x y . x$, $I \equiv \lambda x . x$
- S : applique une fonction à deux résultats, K : retourne une constante, I : identité
- S, K, I suffisent pour représenter tout λ -terme fermé.

5. Correspondance SKI :

- Règles de transformation T pour convertir λ -termes $\rightarrow$ SKI.
- Exemples : $\lambda x.x(\lambda y.y) \rightarrow S I (K I)$, $\lambda x y.y x \rightarrow S (K (S I)) (S (K K) I)$
- Sens inverse : SKI $\rightarrow$ λ -calcul via β -réduction.

6. Réduction SKI :

- Règles élémentaires : $SMNP \rightarrow MP(NP)$, $KMN \rightarrow M$, $IM \rightarrow M$
- Compatibilité avec λ -calcul, substitutions triviales car pas de variables libres.

7. Combinateurs BCKW :

- $B \equiv \lambda f g x . f(gx)$, $C \equiv \lambda f x y . f y x$, $K \equiv \lambda x y . x$, $W \equiv \lambda x y . x y y$
- $I \equiv WK$, S et K exprimables via B, C, W
- Conversion $\lambda \rightarrow SKI \rightarrow BCKW$ possible

8. Combinateur de point fixe :

- Tout λ -terme F a un point fixe N tel que $F N =_{\beta} N$
- Curry-Rosser : $Y \equiv \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$
- Turing : $\Theta \equiv (\lambda x y.y(xxy))(\lambda x y.y(xxy))$
- Exemple : $F \equiv \lambda a b . a$, point fixe $N = (\lambda x b . xx)(\lambda x b . xx)$ vérifié via β -réduction

Chapitre 3 : Théorie des types

1. Introduction :

- Types associent domaines et codomaines aux λ -termes.
- Approches : explicite (Church) ou implicite (Curry).
- Typage $\rightarrow$ forte normalisation, évite paradoxes comme Russell.

2. Paradoxe de Russell :

- Ensemble $R = \{x \mid x \notin x\} \rightarrow \lambda$ -calcul : $R \equiv \lambda x. \text{NOT}(xx)$
- $RR \rightarrow \beta \text{NOT}(RR) \rightarrow$ contradiction $\rightarrow$ typage nécessaire.

3. Types :

- Types atomiques : $A, B, \dots$
- Types fonctionnels : $\sigma \rightarrow \tau$, associativité à droite.

4. Types paramétriques :

- Variables de types : $a, b, \dots$
- Type fermé = seulement atomiques, ouvert = contient variables.
- Exemples : $N \rightarrow N$, $(N \rightarrow N) \rightarrow H$, $a \rightarrow a$.

5. Variables typées :

- $x^\sigma, y^\tau, \dots$
- Cohérence : une variable $\rightarrow$ un seul type.

6. λ -calcul simplement typé (Church) :

- Typage explicite : $(\lambda x^\sigma. M)^\sigma \rightarrow \tau$
- Application : $M^{\{\sigma \rightarrow \tau\}} N^\sigma \rightarrow (M N)^\tau$

7. λ -calcul simplement typé (Curry) :

- Typage implicite via environnement Γ : $\Gamma \vdash M : \sigma$
- Types déduits automatiquement, pas écrits dans le terme.

8. Comparaison Church vs Curry :

- Church : typage explicite, plusieurs écritures possibles selon type.
- Curry : typage implicite, une seule écriture générale.

9. Propriétés :

- Substitution : type préservé si compatible.
- α -équivalence : comme λ -calcul non typé.
- β -réduction : respecte types.
- η -réduction : peut changer le type.

10. Règles de typage :

- Variable : type dans contexte.
- Abstraction : $\sigma \rightarrow \tau$.
- Application : fonction appliquée à argument du bon type.
- Exemples : $\lambda xyz. x(yz)$, $\lambda x. xx$ non typable, $\lambda fx. f(fx) : (\sigma \rightarrow \sigma) \rightarrow \sigma \rightarrow \sigma$.

Chapitre 4 : Théorème de Curry-Howard

1. Introduction

- **Curry-Howard** : correspondance λ -calcul typé $\leftrightarrow$ logique intuitionniste.
- **Idées principales** :
 - Une preuve = un programme
 - Une proposition = un type
 - Une proposition prouvable $\Leftrightarrow$ programme qui s'exécute sans erreur

- **Intérêt** : vérification automatique des programmes, lien maths-informatique.

2. λ -calcul typé

2.1 Types produit et somme

- **Type produit (paire)** : $\sigma \times \tau$
 - Construction : (M, N) de type $\sigma \times \tau$
 - Projections :
 - $\pi_1(M, N)^{\{\sigma \times \tau\}} \rightarrow_{\beta} M^{\sigma}$
 - $\pi_2(M, N)^{\{\sigma \times \tau\}} \rightarrow_{\beta} N^{\tau}$
- **Type somme (variant)** : $\sigma + \tau$
 - Construction :
 - $\text{in}_1^{\tau}(M) : \text{type } \sigma + \tau$
 - $\text{in}_2^{\tau}(M) : \text{type } \tau + \sigma$
 - Élimination (analyse de cas) :
 - $\text{case}(\text{in}_1^{\tau}(M); x.N; y.P) \rightarrow_{\beta} N[x := M]$
 - $\text{case}(\text{in}_2^{\tau}(M); x.N; y.P) \rightarrow_{\beta} P[y := M]$

2.2 Règles de typage

- Paire $\rightarrow$ construit (M, N)
- $\pi_1 / \pi_2 \rightarrow$ extraire composantes
- $\text{in}_1 / \text{in}_2 \rightarrow$ choisir un cas
- $\text{case} \rightarrow$ traiter les deux cas possibles

Priorité des opérateurs : $\times$ et $+$ $\rightarrow$ $>$ $\rightarrow$

- $\sigma \times \tau \rightarrow U \equiv (\sigma \times \tau) \rightarrow U$
- $\sigma \rightarrow \tau \times U \equiv \sigma \rightarrow (\tau \times U)$
- $\sigma + \tau \rightarrow U \equiv (\sigma + \tau) \rightarrow U$
- $\sigma \rightarrow \tau + U \equiv \sigma \rightarrow (\tau + U)$

Exemples :

- $\lambda xyz. x(y, z)$
- $\lambda x. (\pi_2 x, \pi_1 x) : \sigma \times \tau \rightarrow \tau \times \sigma$

3. Logique intuitionniste – rappels

Concept	Logique classique	Logique intuitionniste
Existence	Non-existence impossible suffit	Il faut construire l'objet
Tiers exclu	$A \vee \neg A$ toujours vrai	Non prouvable
$\neg\neg A \Rightarrow A$	Toujours vrai	Non prouvable

Interprétation propositions	A vrai	A prouvable
$\neg A$	A faux	A contradictoire
$A \vee B$	A ou B vrai	A prouvable ou B prouvable
$A \wedge B$	A et B vrai	A et B prouvables
$A \Rightarrow B$	Si A vrai alors B vrai	Si A prouvable alors B prouvable
$\exists x A(x)$	Il existe x	On peut construire x
$\forall x A(x)$	Pour tout x	Pour tout x, prouvable

Exemple constructif : $\exists x, y$ irrationnels avec x^y rationnel

- Non-constructif : raisonnement par cas sans exemple précis
- Constructif : $x = \sqrt{2}$, $y = 2 \log_2 3 \rightarrow x^y = 3$

Règles principales :

- Axiome (Ax) : $\Gamma, \phi \vdash \phi$
- Introduction (I) : $\wedge, \vee, \Rightarrow$
- Élimination (E) : $\wedge, \vee, \Rightarrow$

4. Correspondance Curry-Howard

- **Théorème :** preuves intuitionnistes $\leftrightarrow$ λ -termes typés dans $\lambda\pi$ -calcul.
- **Correspondances clés :**

Logique intuitionniste	$\lambda\pi$ -calcul typé
Axiome	Variable (var)
Preuve	λ -terme
Proposition	Type
$(\wedge I)$	Paire
$(\vee I1), (\vee I2)$	in1, in2
$(\Rightarrow I)$	Abstraction (abs)
$(\wedge E1), (\wedge E2)$	$\pi1, \pi2$

$(\Rightarrow E)$	Application (app)
$(\vee E)$	case

Exemples de propositions $\rightarrow$ types

- $\phi \Rightarrow \psi \rightarrow \sigma \rightarrow \tau$
- $\phi \wedge \psi \rightarrow \sigma \times \tau$
- $\phi \vee \psi \rightarrow \sigma + \tau$

Exemple 1 : $\phi \Rightarrow \psi \Rightarrow \phi$

- $\lambda\pi$ -calcul typé : $\lambda x^\sigma y^\tau. x : \sigma \rightarrow \tau \rightarrow \sigma$

Exemple 2 : $\lambda xy. \pi_1(y, x)$

- Type : $\sigma \times \tau \rightarrow \tau \times \sigma$

Remarque : seules les propositions intuitionnistes prouvables ont un λ -terme typable correspondant.

MAGUEMOUN SAMY